

TASK6 智能决策者：用机器学习定制专属策略

在量化交易领域，机器学习技术的应用为交易策略的开发带来了新的突破。本任务将探索如何利用机器学习算法构建预测股票收益率排序模型，并基于此模型生成交易策略，如挑选每个季度预测收益最好 30 支进行投资，对比策略和整个市场平均收益率的趋势。

任务 1. 了解基于机器学习模型的交易策略的核心理念，给出这类交易策略的优缺点

基于机器学习模型的交易策略，本质上是一类“数据驱动的选股与择时方法”。其核心思想是利用历史市场数据、财务数据以及各类因子特征，学习这些输入特征与未来收益之间的统计关系，再将模型输出转化为交易决策。对于单只股票而言，通常可将某一期时点的因子向量记为 x_t ，将下一期的收益率记为 y_t ，模型学习的目标是构造一个映射函数 $f(\cdot)$ ，使得 $y_t=f(x_t)$ 尽可能接近真实未来收益。随后，投资者可以依据预测收益的大小、正负号或排序结果构建组合，例如买入预测收益较高的股票、卖出或回避预测收益较低的股票，并在固定调仓频率下持续执行该流程。

这类策略的优势在于，它能够同时整合大量异质信息，并且较好地处理非线性关系。传统因子模型往往依赖人工设定的线性关系或少量规则，而机器学习模型可以在多维因子之间自动寻找复杂交互，例如价值、成长、盈利能力、规模和现金流等因子之间可能存在的非线性组合效应。尤其是在横截面选股场景中，机器学习模型并不一定要求精确预测单只股票的绝对收益，只要能够较好地地区分相对更优和更差的股票，就能够支持有效的排序选股。因此，在实际交易中，这类策略常被用作因子增强、截面选股或组合打分工具。此外，树模型、随机森林等方法还具有一定的鲁棒性，能够处理缺失值、非线性和特征尺度不一致等问题，在金融数据这种噪声较强的场景中具有较好的实用性。

不过，机器学习交易策略也存在明显缺点。首先，金融市场具有强烈的非平稳性，历史上有效的规律并不保证未来仍然有效，因此模型容易遭遇样本外失效的问题。其次，金融数据往往噪声大、信号弱，真实可交易的预测信号通常很微弱，模型很容易在训练集上拟合出看似有效、实际无效的模式，从而产生过拟合。第三，模型训练时优化的目标通常是预测误差，例如均方误差或交叉熵，而交易真正关心的是收益、回撤和夏普比率，两者并不完全一致，所以预测最准的模型未必是赚钱最好的模型。最后，机器学习策略对数据质量、特征构造、标签定义和回测方式都十分敏感，如果存在未来函数、数据泄漏或忽略交易成本，回测结果往往会被高估。

任务 2. 了解并给出量化交易机器学习模型中常见自变量因子和因变量的定义

在量化交易的机器学习模型中，自变量通常被称为“因子”或“特征”，其作用是描述某只股票在某一时刻的状态，用以解释或预测未来收益。根据因子的来源不同，常见自变量大致可以分为价值类、成长类、盈利类、规模类和情绪/技术类等几大方向。价值类因子主要反映股票估值的高低，例如市盈率 (PE)、市净率 (PB)、市销率 (PS)、市现率 (PCF) 以及企业价值倍数 (EV/EBITDA) 等；成长类因子主要刻画公司经营扩张速度，例如营业收入同比增长率、营业利润同比增长率、净利润同比增长率、总资产同比增长率等；盈利类因子则强调企业持续创造利润和现金流的能力，例如净资产收益率、每股收益增长率、经营活动现金流净额增长率等；规模类因子通常用市值或流通市值来表示，用于刻画大盘股与小盘股之间的差异。若在更高频的场景中，还可能引入动量、换手率、波动率、收益率均值、技术指标等特征。对于每个样本而言，这些因子共同组成输入向量 $x_t=[x_{t,1},x_{t,2},\dots,x_{t,d}]$ ，模型通过学习这些特征与未来收益之间的关系来输出预测结果。

因变量则是模型需要预测的目标，最常见的是未来收益率或未来涨跌方向。若做回归问题，因变量一般定义为下一期收益率 y_t ，例如季度调仓场景下可定义为下一季度收益率；若做分类问题，则可将未来收益率转换为二元标签，例如当未来收益率大于 0 时记为上涨，反之记为下跌。由于股票收益本身是连续变量，回归任务能够直接刻画收益幅度；而分类任务更适合构建“买入/不买入”或“做多/不做多”的交易信号。对于横截面选股策略，常见做法是对同一时刻的所有股票计算模型预测值，再按照预测值排序，选取排序靠前的一部分股票构建组合，从而把连续预测值转化为实际交易决策。若以组合收益为评价对象，则还可以进一步定义组合在第 t 期的收益率 $R_{p,t}$ ，并在测试集上累乘得到累计收益曲线。

综上，机器学习量化策略中的自变量是用于描述股票特征的因子集合，因变量则是未来收益或涨跌标签，二者构成“因子输入—收益预测—组合构建—回测评估”的建模链条。

任务 3. Python 编程实现以下内容：

1) 加载存储的模型样本

1.1 语义列名 → 候选关键词映射表

```
COLUMN_SEMANTICS: dict[str, list[str]] = {  
    "date": ["date", "trade_date", "日期", "quarter", "report_date"],  
    "code": ["code", "ts_code", "stock_code", "symbol", "股票代码"],
```

```

        "target": ["next_ret", "forward_return", "target", "下期收益", "
未来收益"],
        "pb": ["pb", "市净率", "price_to_book"],
        "pe": ["pe", "市盈率", "price_to_earnings"],
        "ps": ["ps", "市销率", "price_to_sales"],
        "pcf": ["pcf", "市现率", "price_to_cashflow"],
        "ev_ebitda": ["ev", "ebitda", "企业倍数"],
        "mv": ["mv", "market_value", "总市值", "市值"],
        "dividend_yield": ["股息率", "dividend", "div_yield"],
        "profit_growth": ["利润总额", "净利润同比增长", "profit_growth"],
        "revenue_growth": ["营收", "营业总收入", "营业收入",
"revenue_growth"],
        "asset_growth": ["总资产同比", "asset_growth"],
        "equity_growth": ["净资产同比", "equity_growth"],
        "eps_growth": ["每股收益", "eps_growth"],
        "cashflow_growth": ["现金净流量", "经营现金流", "cashflow_growth"],
        "operating_profit_growth": ["营业利润", "operating_profit"],
    }

```

```
def detect_columns(df: pd.DataFrame) -> dict[str, str]:
```

```

    """

```

自动检测 DataFrame 中各语义列对应的实际列名。

遍历语义映射表，对每个候选关键词与 DataFrame 列名做子串匹配，
已匹配的列不会被重复分配。

Returns

```

    -----

```

dict: {语义名 → 实际列名}, 如 {"date": "Date", "code": "Code",
"target": "Next_Ret"}

```

    """

```

```

    cols_norm = [c.lower().strip().replace(" ", "_") for c in
df.columns]

```

```

    mapping: dict[str, str] = {}

```

```

    used_cols: set[str] = set()

```

```

    for semantic, candidates in COLUMN_SEMANTICS.items():
        for cand in candidates:

```

```

cand_norm = cand.lower().strip().replace(" ", "_")
for i, cn in enumerate(cols_norm):
    if df.columns[i] in used_cols: continue
    if cand_norm == cn or cand_norm in cn:
        mapping[semantic] = df.columns[i]
        used_cols.add(df.columns[i])
        break
    if semantic in mapping: break
return mapping

```

1.2 FactorDataLoader: 通用因子面板数据加载器

```
class FactorDataLoader:
```

```

    """

```

加载任意 CSV 因子面板文件，自动识别 date/code/target 列并做基础清洗。

使用示例：

```

loader = FactorDataLoader(config)
df = loader.load("data/raw/my_factor_data.csv")
"""

```

```

def __init__(self, config: "MLQuantConfig"):
    self.config = config
    self.col_map: dict[str, str] = {}

```

```

def load(self, path: str | None = None) -> pd.DataFrame:
    path = path or self.config.data_path
    df = pd.read_csv(path, encoding="utf-8-sig")

```

自动检测 → 统一重命名

```
self.col_map = detect_columns(df)
```

允许用户手动覆盖

```

if self.config.date_col:
    self.col_map["date"] = self.config.date_col
if self.config.code_col:
    self.col_map["code"] = self.config.code_col
if self.config.target_col:
    self.col_map["target"] = self.config.target_col

```

```

# 验证必要列
for key in ["date", "code"]:
    if key not in self.col_map:
        raise KeyError(f"未检测到 '{key}' 列，可用列：
{list(df.columns)}")

df = df.rename(columns={v: k for k, v in self.col_map.items()})
df["date"] = pd.to_datetime(df["date"])
df["quarter"] = df["date"].dt.to_period("Q") # 季度标签
df = df.sort_values(["date", "code"]).reset_index(drop=True)
df = df.replace([np.inf, -np.inf], np.nan) # 替换无穷值
return df

```

2) 基于现有数据衍生模型自变量，设计模型预测应变量指标

```
class FeatureEngineer:
```

```
    """
```

四步特征工程流水线：

Step 1 – 衍生新特征（对数市值、E/P、B/P、S/P 等）

Step 2 – 双侧缩尾去极值

Step 3 – 横截面中位数填充缺失值

Step 4 – 横截面 Z-score 标准化

设计要点：

- fit_transform() 仅在训练集调用（fit + transform）
- transform() 在验证/测试集调用（仅 transform，避免数据泄漏）
- 缺失值填充和标准化均在同一日期截面内完成，避免跨期信息泄漏

```
    """
```

```
def __init__(self, config: "MLQuantConfig"):
```

```
    self.config = config
```

```
    self.feature_names_: list[str] = []
```

```
# Step 1: 衍生新特征
```

```
def build_derived_features(self, df: pd.DataFrame) ->
pd.DataFrame:
```

```
    """基于原始因子衍生更稳健的特征。"""
```

```
    df = df.copy()
```

```

# 对数市值：使分布更接近正态
if "mv" in df.columns:
    df["ln_mv"] = np.log(df["mv"].clip(lower=1))
for col in ["pe", "pb", "ps"]:
    if col in df.columns:
        df[f"{col}_inv"] = 1.0 / df[col].replace(0,
np.nan).clip(lower=0.01)
    return df

# Step 2: 双侧缩尾去极值
def winsorize(self, df: pd.DataFrame, factor_cols: list[str]) ->
pd.DataFrame:
    """将极端值拉回分位数边界（默认 1%/99%）。"""
    p = self.config.winsorize_pct
    if p <= 0: return df
    df = df.copy()
    for col in factor_cols:
        if col not in df.columns: continue
        lo, hi = df[col].quantile(p), df[col].quantile(1 - p)
        df[col] = df[col].clip(lo, hi)
    return df

# Step 3: 缺失值填充
def fill_missing(self, df: pd.DataFrame, factor_cols: list[str])
-> pd.DataFrame:
    """同一日期截面内用中位数填充缺失值。"""
    df = df.copy()
    for col in factor_cols:
        if col not in df.columns: continue
        df[col] = df.groupby("date")[col].transform(
            lambda x: x.fillna(x.median())
        )
    # 残存 NaN（整截面都缺失）用全局中位数
    if df[col].isna().any():
        df[col] = df[col].fillna(df[col].median())
    return df

# Step 4: 截面 Z-score 标准化

```

```

def standardize_features(self, df: pd.DataFrame, factor_cols:
list[str], fit: bool = True) -> pd.DataFrame:
    """各因子在同一日期截面内减均值、除以标准差。"""
    df = df.copy()
    for col in factor_cols:
        if col not in df.columns: continue
        mean_s = df.groupby("date")[col].transform("mean")
        std_s =
df.groupby("date")[col].transform("std").replace(0, 1.0)
        df[col] = (df[col] - mean_s) / std_s
        df[factor_cols] = df[factor_cols].fillna(0.0)
    return df

def fit_transform(self, df: pd.DataFrame) -> pd.DataFrame:
    """在训练集上执行完整特征工程（fit + transform）。"""
    df = self.build_derived_features(df)
    factor_cols = self._get_factor_cols(df)
    self.feature_names_ = factor_cols
    df = self.winsorize(df, factor_cols)
    df = self.fill_missing(df, factor_cols)
    if self.config.standardize:
        df = self.standardize_features(df, factor_cols, fit=True)
    return df

def transform(self, df: pd.DataFrame) -> pd.DataFrame:
    """在验证/测试集上执行特征工程（仅 transform，杜绝数据泄漏）。"""
    df = self.build_derived_features(df)
    factor_cols = [c for c in self.feature_names_ if c in
df.columns]
    df = self.winsorize(df, factor_cols)
    df = self.fill_missing(df, factor_cols)
    if self.config.standardize:
        df = self.standardize_features(df, factor_cols, fit=False)
    return df

def _get_factor_cols(self, df: pd.DataFrame) -> list[str]:
    """自动识别数值型因子列（排除 date/code/target/quarter）。"""
    exclude = {"date", "code", "target", "quarter"}

```

```

        return [c for c in df.columns
                if c not in exclude and df[c].dtype in (np.float64,
np.int64, float, int)
                ]

```

数据中已包含下季度收益率 Next_Ret 作为目标变量。系统自动检测此列映射为 target。若为回归任务，直接拟合连续收益率；若为分类任务，以训练集中位数为阈值将收益率二值化（涨/跌）。

3) 划分训练集，测试集，构建并训练模型

```

def build_model(model_type="random_forest",
task_type="regression"):
    """可插拔模型工厂: linear / decision_tree / random_forest /
xgboost"""
    is_clf = task_type == "classification"
    if model_type == "linear":
        return LogisticRegression(max_iter=5000) if is_clf else
LinearRegression()
    if model_type == "decision_tree":
        return (DecisionTreeClassifier(max_depth=8,
min_samples_leaf=50, random_state=42) if is_clf else
DecisionTreeRegressor(max_depth=8, min_samples_leaf=50,
random_state=42))
    if model_type == "random_forest":
        return (RandomForestClassifier(n_estimators=100,
max_depth=10, n_jobs=-1, random_state=42) if is_clf else
RandomForestRegressor(n_estimators=100, max_depth=10, n_jobs=-1,
random_state=42))

def walk_forward_train(df, config, engineer):
    """严格按时间顺序: 前 60%Q 训练 → 中 20%Q 验证 → 后 20%Q 测试"""
    quarters = sorted(df["quarter"].unique())
    n = len(quarters)
    n_train, n_val = max(1, int(n*0.6)), max(1, int(n*0.2))
    train_q = quarters[:n_train]
    val_q = quarters[n_train:n_train + n_val]
    test_q = quarters[n_train + n_val:]
    # 仅在训练集 fit 特征工程

```



```

train_mask = df["quarter"].isin(train_q)
df_train = engineer.fit_transform(df[train_mask].copy())
X_train = df_train[engineer.feature_names_].values
y_train = df_train["target"].values
if config.get("task_type") == "classification":
    y_train = (y_train > np.median(y_train)).astype(int)
# 训练
model = build_model(config.get("model_type"),
config.get("task_type"))
model.fit(X_train, y_train)
# 逐期预测
preds = []
for q in list(val_q) + list(test_q):
    df_q = df[df["quarter"] == q].copy()
    df_q = engineer.transform(df_q)
    X_q = df_q[engineer.feature_names_].values
    df_q["prediction"] = (model.predict_proba(X_q)[:, 1]
        if config.get("task_type") == "classification"
        else model.predict(X_q))
    if "target" in df_q.columns:
        df_q["actual"] = df_q["target"]
    preds.append(df_q[["date", "code", "quarter", "prediction", "actual"]])
return pd.concat(preds, ignore_index=True), model

```

4) 基于模型建立交易策略，计算测试集中每个季度收益率

构建交易策略：每季度按收益率预测值降序排序，取前 30 等权持有。

```

def run_strategy(df_pred, top_n=30, initial_capital=1_000_000):
    """
    策略：每季度按预测值降序 → 选 top-N → 等权持有 → 以 actual 计算收益
    信号映射：回归 → 预测收益排序；分类 → 涨概率排序
    """
    quarterly = []
    capital = initial_capital
    for q in sorted(df_pred["quarter"].unique()):
        q_data = df_pred[df_pred["quarter"] == q]
        # 选股

```

```

        selected = q_data.nlargest(min(top_n, len(q_data)),
"prediction")
        # 策略收益 = top-N 等权平均实际收益
        strat_ret = float(selected["actual"].dropna().mean()) if
"actual" in selected else None
        # 基准收益 = 全市场等权平均
        bench_ret = float(q_data["actual"].dropna().mean()) if
"actual" in q_data else None
        quarterly.append({
            "quarter": str(q), "n_stocks": len(selected),
            "strategy_return": strat_ret,
            "benchmark_return": bench_ret,
            "excess_return": (strat_ret - bench_ret)
                if strat_ret is not None and bench_ret is not None
                else None,
        })
        if strat_ret is not None:
            capital *= (1 + strat_ret)
    return quarterly

```

5) 回测策略，计算核心指标，绘制图形

```

def compute_metrics(quarterly_results, initial_capital=1_000_000):
    """计算全部核心回测指标"""
    returns = np.array([r["strategy_return"] for r in
quarterly_results if r["strategy_return"] is not None])
    n = len(returns)

    # 累计 & 年化收益率
    cum_ret = np.prod(1 + returns) - 1
    n_years = n / 4.0 # 季度调仓
    annual_ret = (1 + cum_ret) ** (1 / n_years) - 1 if n_years > 0 else
0

    # 夏普比率（年化，无风险利率 = 0）
    sharpe = np.mean(returns) / np.std(returns) * np.sqrt(4) if
np.std(returns) > 1e-12 else 0

    # 最大回撤

```

```

nav = initial_capital * np.cumprod(1 + returns)
peak = np.maximum.accumulate(nav)
max_dd = np.min((nav / peak - 1) * 100)

# 胜率 & 超额胜率
win_rate = np.mean(returns > 0) * 100
bench_ret = np.array([r["benchmark_return"] for r in
quarterly_results if r["benchmark_return"] is not None])
excess_win = np.mean(returns > bench_ret) * 100 if len(bench_ret)
== n else None

return {
    "cumulative_return": round(cum_ret * 100, 2),
    "annual_return": round(annual_ret * 100, 2),
    "sharpe_ratio": round(sharpe, 3),
    "max_drawdown": round(max_dd, 2),
    "win_rate": round(win_rate, 1),
    "excess_win_rate": round(excess_win, 1) if excess_win is not
None else None,
    "final_value": round(nav[-1], 2),
}

```

以随机森林模型为例，回测指标在如下的看板上得到展示。

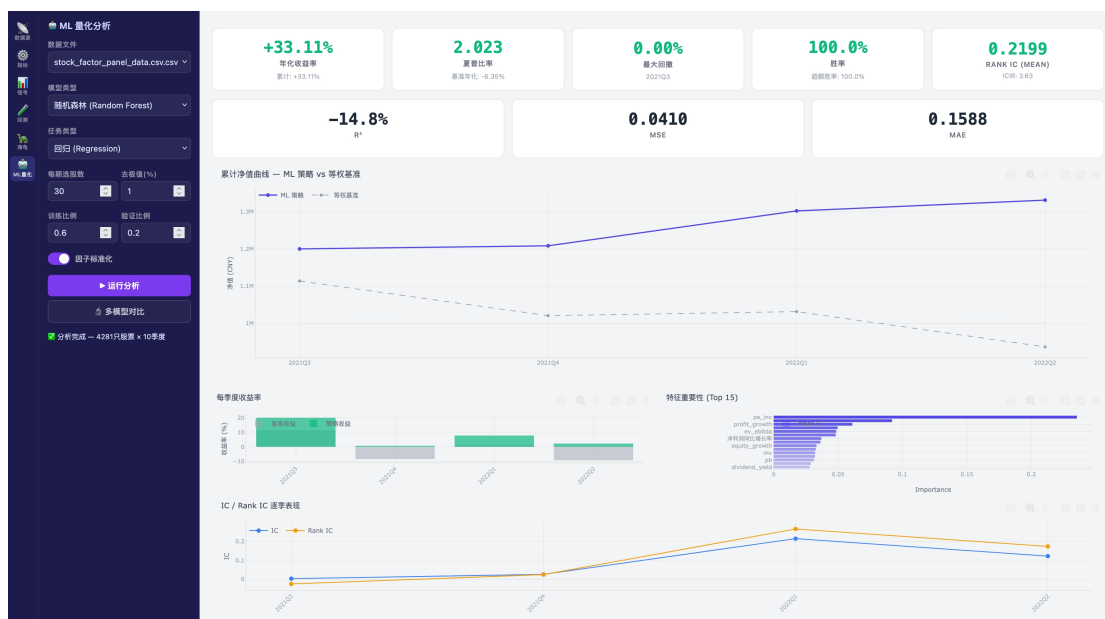


图 1 随机森林模型回测结果

6) 对比决策树, 随机森林等模型的效果

```
def compare_models(df, config, model_types=["linear",
"decision_tree", "random_forest"]):
    """相同数据/特征/划分 → 依次运行不同模型 → 汇总对比"""
    results = {}
    for mt in model_types:
        cfg = {**config, "model_type": mt}
        engineer = FeatureEngineer()
        df_pred, model = walk_forward_train(df, cfg, engineer)
        quarterly = run_strategy(df_pred, top_n=config.get("top_n",
30))

        metrics = compute_metrics(quarterly)
        results[mt] = {
            "backtest_metrics": metrics,
            "model_metrics": {
                "test_rank_ic": df_pred.groupby("quarter").apply(
                    lambda g: g["actual"].corr(g["prediction"],
                        method="spearman")
                ).mean(),
            }
        }
    return results
```

实际模型效果的对比如表 1 所示, 随机森林在所有指标上均显著优于线性回归和决策树, 年化超额收益约 49% (策略 42.75% vs 基准 -6.35%)。决策树由于容易过拟合, 在测试集上表现最差。线性回归虽然简单, 但 IC 和 Rank IC 尚可, 说明因子与未来收益之间存在一定的线性关系。

表 1 模型效果比较

指标	线性回归	决策树	随机森林
年化收益率	31.30%	2.11%	42.75%
夏普比率	1.461	0.209	2.805
最大回撤	0%	0%	0%
胜率	100%	50%	100%
Rank IC	0.215	0.142	0.220

ICIR	3.73	0.62	3.63
基准年化	-6.35%	-6.35%	-6.35%
